

Self-Practice Evidence for Programming

Võ Thiện Trí Nhân

October 30, 2025 – November 1, 2025

1 Introduction

Greetings! My name is Võ Thiện Trí Nhân, a student studying Information Technology at Ho Chi Minh City University of Science. I wrote this report to monitor my self-practice progress in learning programming.

My job is to solve problems on online platforms such as LeetCode or HackerRank (Codeforces is excluded from the list) and record solved exercises in this report. The online platform I mainly used to solve problems is Leetcode, one of the most well-known websites for practicing programming and mindset for interviews. The reason I chose this website is because of familiarity; I used to find exercises to practice on this website. To be honest, I couldn't solve most of them.

This is the end of the introduction. I hope this report will be accepted. Hope the readers have a good day.

2 Problems

2.1 Problem 1

- Name: Palindrome Number
- Link: [Leetcode - Palindrome Number](#)
- Level: Easy

Solution

```
class Solution {
public:
    bool isPalindrome(long int x) {
        long int store = x;
        int remainder = 0;
        long int total = 0;
        int digits = 0;

        if (store < 0) {
            return false;
        }
        else {
            while (store > 0)
            {
                store /= 10;
                digits++;
            }
        }

        store = x; // Make another copy of x to store
```

```

    for (int i = digits; i > 0; i--) {
        remainder = store % 10;
        store /= 10;
        total += remainder * pow(10, i-1);
    }

    if (x == total) {
        return true;
    }
    else {
        return false;
    }
}
};

```

2.1.1 Description

The problem asks us to determine whether a number is a palindrome or not.

1. Variable Definition:

- **store**: Data type `long int`. Purpose: Get a copy of x .
- **remainder**: Data type `int`. Purpose: Store one digit.
- **total**: Data type `long int`. Purpose: Store the reversed number.
- **digits**: Data type `int`. Purpose: Identify the number of digits.

2. Negative Check: Negative numbers cannot be palindromes (e.g., $-121 \rightarrow 121-$).

3. Count Digits: A while loop finds the total number of digits by repeatedly dividing **store** by 10.

4. Reset Copy: Reset **store** back to the original value x .

```
store = x;
```

5. Reversal Process: A for loop iterates to build the reversed number **total**.

- In each iteration, the last digit is extracted, **store** is reduced, and the digit is placed in its correct positional value in **total**:

$$\text{total} += \text{remainder} \times 10^{(i-1)}$$

6. Final Comparison: Check if the reversed number equals the original.

2.1.2 Test cases

1. **E1: 121**

- (a) Digits calculated as 3.
- (b) Reversal process:
 - $i = 3$: remainder=1, total = $1 \times 10^2 = 100$.
 - $i = 2$: remainder=2, total = $100 + 2 \times 10^1 = 120$.
 - $i = 1$: remainder=1, total = $120 + 1 \times 10^0 = 121$.
- (c) Check: $121 == 121$. **Return true.**

2. **E2: 12321**

- (a) Digits calculated as 5.
- (b) Reversal process:
 - $i = 5$: remainder=1, total = $1 \times 10^4 = 10000$.
 - $i = 4$: remainder=2, total = $10000 + 2 \times 10^3 = 12000$.
 - $i = 3$: remainder=3, total = $12000 + 3 \times 10^2 = 12300$.
 - $i = 2$: remainder=2, total = $12300 + 2 \times 10^1 = 12320$.
 - $i = 1$: remainder=1, total = $12320 + 1 \times 10^0 = 12321$.
- (c) Check: $12321 == 12321$. **Return true.**

3. **E3: 1255**

- (a) Digits calculated as 4.
- (b) Reversal process yields total = 5521.
- (c) Check: $1255 \neq 5521$. **Return false.**

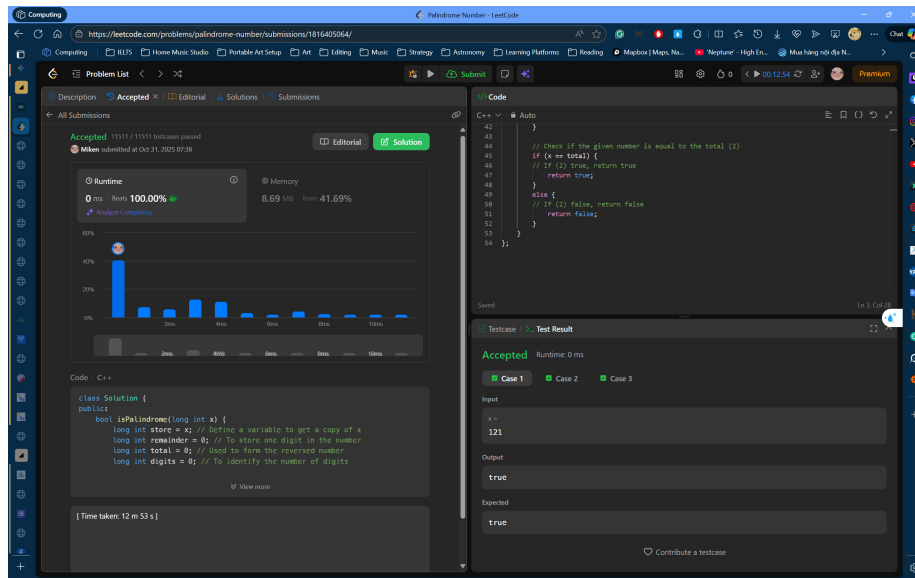


Figure 1: Problem 1: Palindrome Number

2.2 Problem 2

- Name: Roman to Integer
- Link: [Leetcode - Roman to Integer](#)
- Level: Easy

2.2.1 Solution

```
class Solution {
public:
    int romanToInt(string s) {
        int total = 0;

        char one = 'I';
        char five = 'V';
        char ten = 'X';
        char fifty = 'L';
        char hundred = 'C';
        char five_hundred = 'D';
        char one_thousand = 'M';

        int i = 0;
```

```

while (i < s.length()) {
    if (i + 1 > s.length()) {
        break;
    }
    else {
        if (s[i] == one) {
            if (s[i + 1] == five) {
                total += 5 - 1; i = i + 2;
            }
            else if (s[i + 1] == ten) {
                total += 10 - 1; i = i + 2;
            }
            else if (s[i + 1] == fifty) {
                total += 50 - 1; i = i + 2;
            }
            else if (s[i + 1] == hundred) {
                total += 100 - 1; i = i + 2;
            }
            else if (s[i + 1] == five_hundred) {
                total += 500 - 1; i = i + 2;
            }
            else if (s[i + 1] == one_thousand) {
                total += 1000 - 1; i = i + 2;
            }
            else {
                total += 1; i++;
            }
        }
        else if (s[i] == five) {
            if (s[i + 1] == ten) {
                total += 10 - 5; i = i + 2;
            }
            else if (s[i + 1] == fifty) {
                total += 50 - 5; i = i + 2;
            }
            else if (s[i + 1] == hundred) {
                total += 100 - 5; i = i + 2;
            }
            else if (s[i + 1] == five_hundred) {
                total += 500 - 5; i = i + 2;
            }
            else if (s[i + 1] == one_thousand) {
                total += 1000 - 5; i = i + 2;
            }
            else {
                total += 5; i++;
            }
        }
    }
}

```

```

    }
}
else if (s[i] == ten) {
    if (s[i + 1] == fifty) {
        total += 50 - 10; i = i + 2;
    }
    else if (s[i + 1] == hundred) {
        total += 100 - 10; i = i + 2;
    }
    else if (s[i + 1] == five_hundred) {
        total += 500 - 10; i = i + 2;
    }
    else if (s[i + 1] == one_thousand) {
        total += 1000 - 10; i = i + 2;
    }
    else {
        total += 10; i++;
    }
}
else if (s[i] == fifty) {
    if (s[i + 1] == hundred) {
        total += 100 - 50; i = i + 2;
    }
    else if (s[i + 1] == five_hundred) {
        total += 500 - 50; i = i + 2;
    }
    else if (s[i + 1] == one_thousand) {
        total += 1000 - 50; i = i + 2;
    }
    else {
        total += 50; i++;
    }
}
else if (s[i] == hundred) {
    if (s[i + 1] == five_hundred) {
        total += 500 - 100; i = i + 2;
    }
    else if (s[i + 1] == one_thousand) {
        total += 1000 - 100; i = i + 2;
    }
    else {
        total += 100; i++;
    }
}
else if (s[i] == five_hundred) {
    if (s[i + 1] == one_thousand) {

```

```

        total += 1000 - 500; i = i + 2;
    }
    else {
        total += 500; i++;
    }
}
else if (s[i] == one_thousand) {
    total += 1000; i++;
}
}
}

return total;
}
};

```

2.2.2 Description

The algorithm converts a Roman numeral string into an integer by scanning each character from left to right:

- If the next character has a greater value, it means a subtractive case (e.g., $IV = 5 - 1 = 4$). In this case, the algorithm adds the difference and skips the next character (advances the index by 2).
- Otherwise, it simply adds the value of the current character (advances index by 1).
- This process continues until the end of the string, and the total is returned.

2.2.3 Test cases

1. VII

- $V = 5$, next is I (smaller) $\rightarrow$ add 5 $\rightarrow$ total = 5
- $I = 1$, next is I (equal) $\rightarrow$ add 1 $\rightarrow$ total = 6
- $I = 1$, no next $\rightarrow$ add 1 $\rightarrow$ total = 7

Result: **7**

2. IVII

- $I = 1$, next is V (greater) $\rightarrow$ add $5 - 1 = 4$, skip next $\rightarrow$ total = 4
- $I = 1$, next is I (equal) $\rightarrow$ add 1 $\rightarrow$ total = 5
- $I = 1$, no next $\rightarrow$ add 1 $\rightarrow$ total = 6

Result: **6**

3. MCMVIHD

- M = 1000, next is C (smaller) $\rightarrow$ add 1000 $\rightarrow$ total = 1000
- C = 100, next is M (greater) $\rightarrow$ add $1000 - 100 = 900$, skip next $\rightarrow$ total = 1900
- V = 5, next is I (smaller) $\rightarrow$ add 5 $\rightarrow$ total = 1905
- I = 1, next is I (equal) $\rightarrow$ add 1 $\rightarrow$ total = 1906
- I = 1, next is I (equal) $\rightarrow$ add 1 $\rightarrow$ total = 1907
- I = 1, next is D (greater) $\rightarrow$ add $500 - 1 = 499$, skip next $\rightarrow$ total = 2406

Result: **2406**

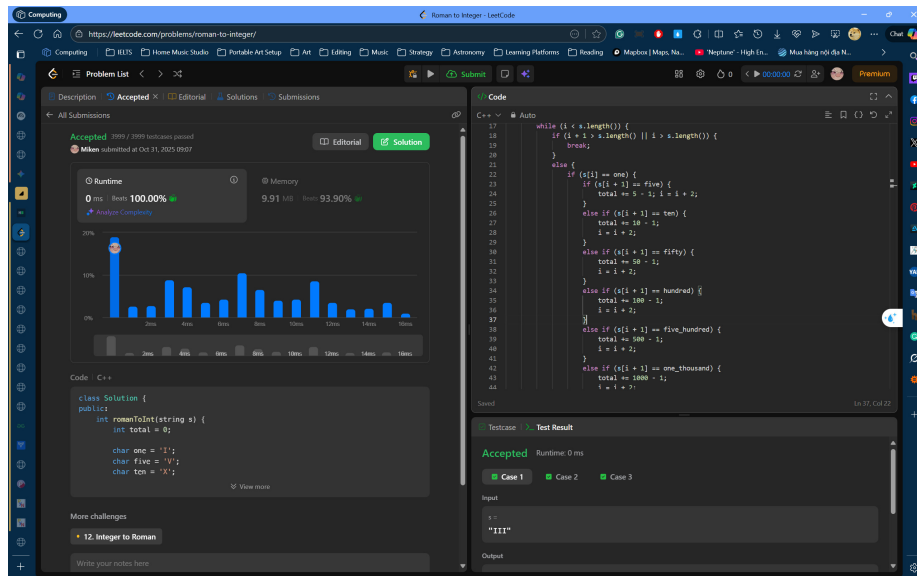


Figure 2: Problem 2: Roman to Integer

2.3 Problem 3

- Name: sqrt(x)
- Link: [Leetcode - sqrt\(x\)](#)
- Level: Easy

2.3.1 Solution

```
class Solution {
public:
    int mySqrt(int x) {
        long int low = 1, high = x, mid, possible;

        if (x == 0 || x == 1) {
            return x;
        }
        else {
            while (low <= high) {
                mid = (low + high) / 2;
                if (mid * mid == x) {
                    return mid;
                }
                else if (mid * mid > x) {
                    high = mid - 1;
                }
                else if (mid * mid < x) {
                    low = mid + 1;
                    possible = mid;
                }
            }

            return possible;
        }
    }
};
```

2.3.2 Description

The algorithm finds the integer square root of a number x using **binary search**:

- If $x = 0$ or $x = 1$, return x immediately.
- Otherwise, set two pointers: $low = 1$ and $high = x$.
- While $low \leq high$:
 - Compute $mid = \frac{low+high}{2}$.
 - If $mid \times mid = x$, return mid .
 - If $mid \times mid > x$, set $high = mid - 1$.
 - If $mid \times mid < x$, set $low = mid + 1$ and store mid as the best possible answer so far (**possible**).
- When the loop ends, return the last stored **possible** answer (the floor of the square root).

2.3.3 Test cases

1. **Input: 49 - Result: 7**
2. **Input: 35677 - Result: 188**
3. **Input: 5 - Result: 2**

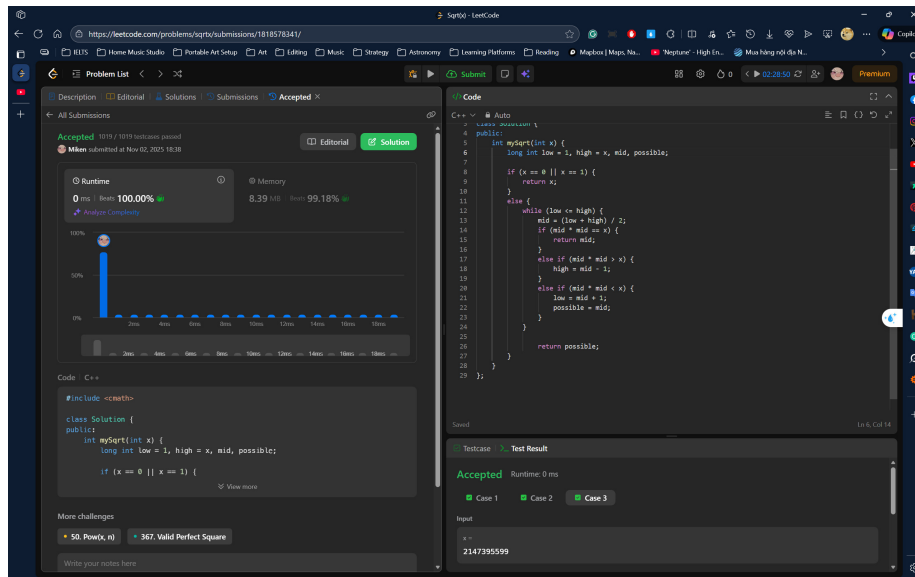


Figure 3: Problem 3: sqrt(x)